

MACD Crossover Strategy Backtesting

Candidate: Nitin Chaturvedi

Role: Junior Python Developer - Remote

Repository: <https://github.com/Sigmanumeric78/MACD-crossover.git>

Project Objective

The objective was to implement the same trading strategy across three different backtesting environments: vectorbt, Nautilus Trader, and MetaTrader 5. The purpose was to demonstrate documentation reading, framework adaptation, reproducible implementation, debugging, and comparison of results across vectorized, event-driven, and platform-native testing workflows.

Strategy Summary

- Instrument: EUR/USD
- Timeframe: 1-hour / H1
- Date range: 2024-01-01 to 2024-12-31
- Strategy type: Long-only MACD crossover
- Entry: MACD line crosses above signal line
- Exit: MACD line crosses below signal line
- MACD settings: Fast EMA 12, Slow EMA 26, Signal EMA 9
- EMA, MACD, and signal line calculations were implemented manually.
- No built-in MACD indicator was used; Python did not use pandas .ewm(); MT5 did not use iMACD.

Implementation Summary

vectorbt

- Loaded cleaned EUR/USD H1 CSV data.
- Generated manual MACD crossover signals.
- Ran a vectorized portfolio backtest.
- Exported results, trades, and equity curve.

Nautilus Trader

- Implemented the same logic as a Nautilus strategy class.
- Used an event-driven backtest approach.
- Processed completed H1 bars.
- Exported result metrics, trades, and reconstructed equity curve.

MetaTrader 5

- Implemented the strategy as an MQL5 Expert Advisor.
- Used manual EMA/MACD logic inside the EA.
- Ran MT5 Strategy Tester manually on Windows.
- Exported the HTML Strategy Tester report.

Results Table

Framework	Data	Trades	Total Return	Sharpe Ratio	Max Drawdown	Notes
vectorbt	REAL_CSV	241	-0.3863%	-0.9415	-0.5761%	Vectorized signal portfolio
Nautilus Trader	REAL_CSV	241	-0.4280%	-6.1039	-0.5740%	Event-driven; equity reconstructed from closed PnL
MetaTrader 5	EURUSD_CUSTOM	230	-0.4325%	-1.23	-0.65%	Manual Strategy Tester run; 16% history quality and 6106 bars

Key Observations

- The strategy was not profitable over the selected EUR/USD H1 2024 period.
- vectorbt and Nautilus produced the same number of trades, but metrics differed due to vectorized versus event-driven execution and equity calculation assumptions.
- MT5 produced fewer trades because the Strategy Tester processed 6106 bars compared to the 6250 rows in the Python processed CSV.
- Differences across frameworks are expected because each engine handles fills, bar processing, spread, execution timing, and reporting differently.

Issues Encountered and Resolved

- Real market data was initially missing; resolved by using ForexSB EURUSD60.csv.
- ForexSB CSV had no header; resolved with a fallback parser.
- vectorbt required handling NUMBA_DISABLE_JIT=1.
- Nautilus required adapting to its event-driven API and FX margin account setup.
- MT5 setup was completed on Windows for reliability.
- MT5 screenshot was not included, but the exported HTML report is included.

AI Tool Usage

AI coding tools were used for scaffolding, implementation planning, debugging, documentation, and cross-framework reasoning. Human verification was performed by running terminal backtests and MT5 Strategy Tester manually. Incorrect or incomplete AI suggestions were validated against actual runtime errors and fixed iteratively.

Final Deliverables

- GitHub repository with code and documentation.
- vectorbt implementation and results.
- Nautilus Trader implementation and results.
- MT5 MQL5 Expert Advisor and exported Strategy Tester report.
- Project README and framework-specific notes.

This summary is intended as a concise attachment for HR/recruitment review. Full implementation details, commands, errors, fixes, and validation notes are documented in the repository README files and MEMORY.md.